

Classification en Machine Learning

Cours Complet - Tous les Modèles

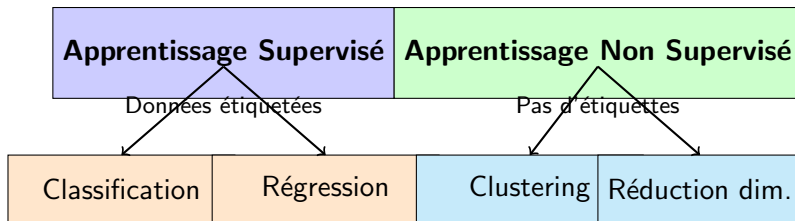
Meyssem Bouzaïen

Année universitaire: 2025 - 2026

Plan du Cours

- 1 Qu'est-ce que la Classification ?
- 2 K-Nearest Neighbors (KNN)
- 3 Arbres de Décision
- 4 Régression Logistique
- 5 Support Vector Machines (SVM)
- 6 Random Forests (Forêts Aléatoires)
- 7 Métriques d'Évaluation
- 8 Comparaison des Modèles
- 9 Mise en Pratique
- 10 Exercices Pratiques
- 11 Bonnes Pratiques et Conclusion

Rappel : Types d'Apprentissage



Aujourd'hui

On se concentre sur la **Classification**

Qu'est-ce que la classification ?

La classification consiste à **prédire une catégorie** (classe) pour de nouvelles observations, à partir d'exemples étiquetés.

Input (Features) :

- Caractéristiques mesurables
- Variables descriptives
- Exemple : taille, poids, couleur...

Output (Classe) :

- Catégorie prédite
- Variable discrète
- Exemple : chat, chien, oiseau

Exemples de Classification

Application	Features (X)	Classes (y)
Email Spam	Mots, expéditeur, liens	Spam / Non-spam
Diagnostic médical	Symptômes, analyses	Malade / Sain
Reconnaissance d'images	Pixels de l'image	Chat / Chien / Oiseau
Crédit bancaire	Revenu, âge, historique	Accordé / Refusé
Sentiment analysis	Texte d'un avis	Positif / Négatif / Neutre
Reconnaissance de fleurs	Taille sépales/pétales	Setosa / Versicolor / Virginica

La classification est partout !

Classification Binaire vs Multi-classes

Classification Binaire

2 classes seulement

- Oui / Non
- Spam / Ham(non Spam)
- Malade / Sain
- Positif / Négatif

Classification Multi-classes

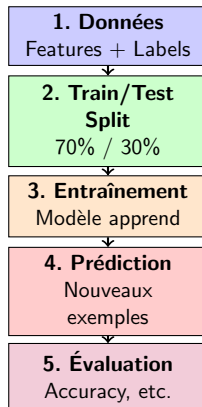
Plus de 2 classes

- 3 types de fleurs Iris
- 10 chiffres (0-9)
- 26 lettres (A-Z)
- 1000 catégories d'objets

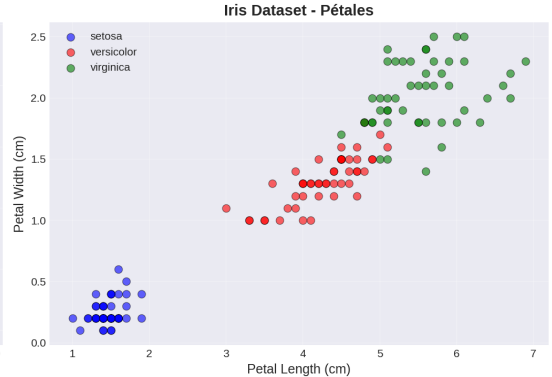
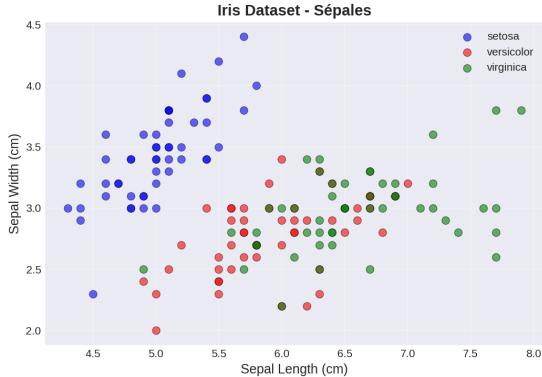
Important

Tous les algorithmes d'aujourd'hui fonctionnent pour les deux !

Le Processus de Classification



Dataset Iris - Notre Exemple



150 fleurs, 3 espèces, 4 features (sécales et pétales)

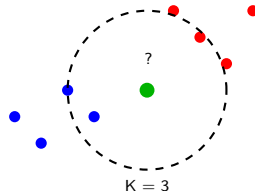
KNN : L'Algorithme le Plus Simple

Idée de base

“Dis-moi qui sont tes voisins, je te dirai qui tu es”

Principe :

- 1 Trouver les K plus proches voisins
- 2 Regarder leur classe
- 3 Vote majoritaire
- 4 Assigner la classe gagnante



KNN : Comment ça marche ? (1/2)

Étape 1 : Calculer la distance

Pour chaque point dans le dataset, calculer la distance avec le nouveau point.

Distance euclidienne (la plus courante) :

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Exemple

Point A = (1, 2) et Point B = (4, 6)

$$d = \sqrt{(4 - 1)^2 + (6 - 2)^2} = \sqrt{9 + 16} = \sqrt{25} = 5$$

KNN : Comment ça marche ? (2/2)

Étape 2 : Trier par distance

Garder les K points les plus proches

Étape 3 : Vote majoritaire

Compter les classes parmi les K voisins

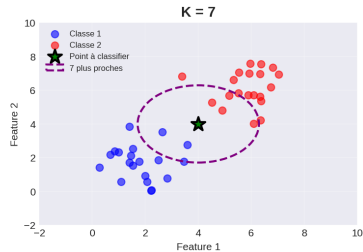
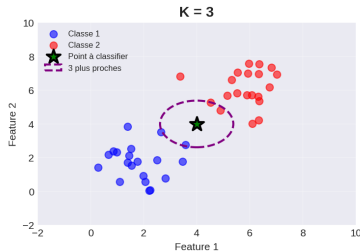
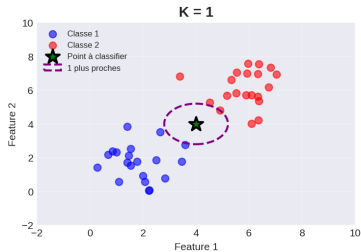
Exemple avec K=5

- Voisin 1 : Classe A (distance : 1.2)
- Voisin 2 : Classe A (distance : 1.5)
- Voisin 3 : Classe B (distance : 1.8)
- Voisin 4 : Classe A (distance : 2.1)
- Voisin 5 : Classe B (distance : 2.3)

Vote : A=3, B=2 → **Classe prédite = A**

Le Paramètre K : Impact

K-Nearest Neighbors: Impact du paramètre K



Conseil

- K trop petit → overfitting
- K trop grand → underfitting
- Valeurs courantes : $K = 3, 5, 7$

KNN avec Scikit-learn

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# 1. Charger les donnees
iris = load_iris()
X, y = iris.data, iris.target

# 2. Train/Test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

# 3. Creer le modele KNN
knn = KNeighborsClassifier(n_neighbors=5)

# 4. Entraîner
knn.fit(X_train, y_train)

# 5. Predire
y_pred = knn.predict(X_test)

# 6. Evaluer
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy*100:.2f}%")
```

KNN : Avantages et Inconvénients

Avantages

- Simple à comprendre
- Pas d'entraînement réel
- Multi-classes naturel
- Pas d'hypothèses

Inconvénients

- Lent (gros datasets)
- Sensible à l'échelle
- Sensible au bruit
- Coûteux en mémoire

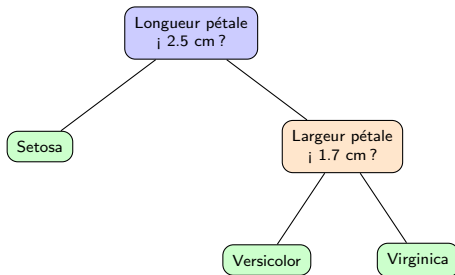
Important

Toujours normaliser les données pour KNN !

Arbres de Décision : Intuition

L'idée

Un arbre de décision pose une série de **questions** sur les features pour arriver à une décision.



Comme un organigramme de décisions !

Comment Construire un Arbre ?

Algorithme

- 1 Choisir la meilleure question
- 2 Diviser les données
- 3 Répéter pour chaque branche
- 4 Arrêter quand tous les exemples ont la même classe

Question clé

Comment choisir la **meilleure** question ?

→ Celle qui sépare le mieux les classes !

Indice de Gini (Simplifié)

Principe

On veut des groupes **purs** (une seule classe par groupe)

$$Gini = 1 - \sum_i p_i^2$$

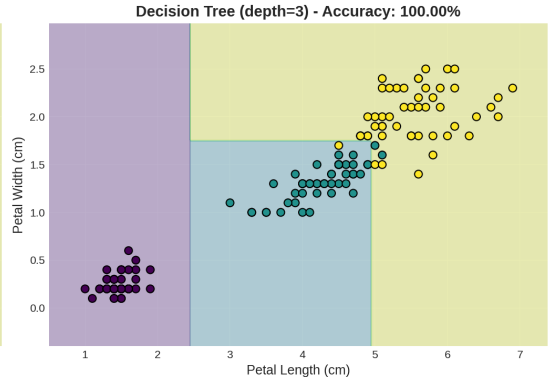
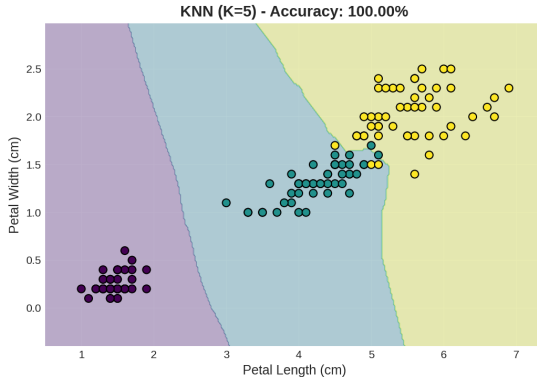
- $Gini = 0 \rightarrow$ parfaitement pur (excellent !)
- $Gini = 0.5 \rightarrow$ mélange 50/50 (mauvais)

Exemple Simple

Groupe A : 10 bleus, 0 rouge $\rightarrow Gini = 0$ (pur !)

Groupe B : 5 bleus, 5 rouges $\rightarrow Gini = 0.5$ (impur)

Frontières de Décision : KNN vs Arbre



KNN : Frontières lisses — **Arbre** : Frontières rectangulaires

Arbres de Décision avec Scikit-learn

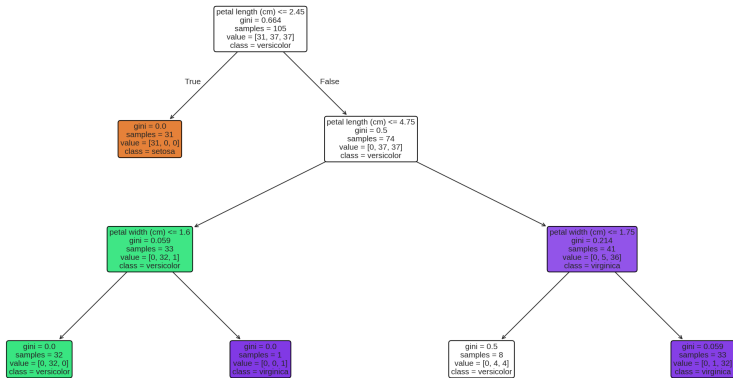
```
1 from sklearn.tree import DecisionTreeClassifier
2
3 # Creer l'arbre
4 tree = DecisionTreeClassifier(
5     max_depth=3,          # Profondeur max
6     min_samples_split=2   # Min pour diviser
7 )
8
9 # Entraîner
10 tree.fit(X_train, y_train)
11
12 # Predire
13 y_pred = tree.predict(X_test)
14
15 # Evaluer
16 accuracy = accuracy_score(y_test, y_pred)
17 print(f"Accuracy: {accuracy*100:.2f}%")
```

Contrôler la complexité

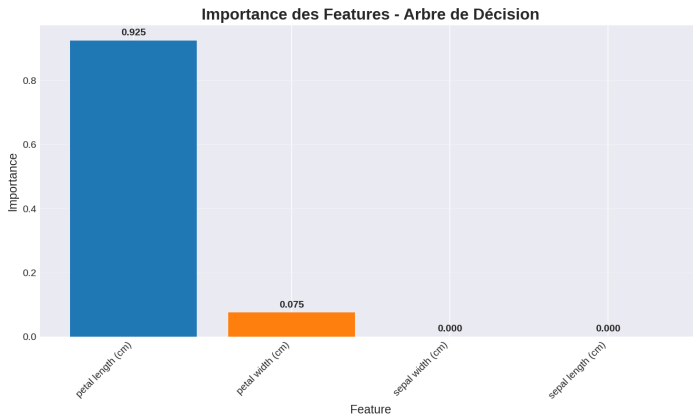
- **max_depth** : Profondeur maximale
 - Trop grand → overfitting
 - Trop petit → underfitting
- **min_samples_split** : Minimum pour diviser
- **min_samples_leaf** : Minimum dans une feuille

L'Arbre Complet Visualisé

Arbre de Décision - Dataset Iris (4 features)



Importance des Features



Les arbres montrent quelles features sont importantes !

Arbres : Avantages et Inconvénients

Avantages

- Facile à visualiser
- Pas de normalisation
- Rapide
- Interprétable
- Features importantes

Inconvénients

- Overfitting
- Instable
- Frontières rigides
- Moins précis

Solution

Random Forests → Plus robustes !

Attention !

Malgré son nom, c'est un algorithme de **CLASSIFICATION**

Principe

Calcule la **probabilité** qu'un exemple appartienne à une classe

Caractéristiques :

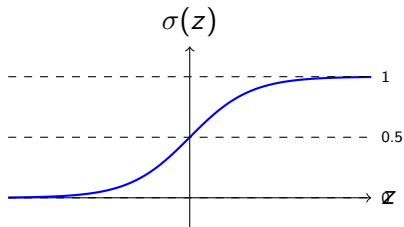
- Excellent pour classification binaire
- Extension multi-classes possible
- Donne des probabilités (0 à 1)
- Rapide et efficace

Fonction Sigmoid

Transformation

Transforme n'importe quelle valeur en probabilité (0 à 1)

$$\sigma(z) = \frac{1}{1+e^{-z}}$$



Comment ça marche ?

- 1 Combinaison linéaire des features : $z = w_1x_1 + w_2x_2 + \dots + b$
- 2 Application sigmoïde : $p = \sigma(z)$
- 3 Décision : Si $p > 0.5 \rightarrow$ Classe 1, sinon Classe 0

Exemple

Features : [taille=1.70, poids=70]

Poids appris : $w_1 = 0.5, w_2 = 0.3, b = -40$

$$z = 0.5 \times 1.70 + 0.3 \times 70 - 40 = 1.85$$

$$p = \sigma(1.85) = 0.86 = 86\%$$

$\rightarrow$ **Classe 1 avec 86% de confiance**

Régression Logistique : Code

```
from sklearn.linear_model import LogisticRegression

# Creer le modele
log_reg = LogisticRegression(max_iter=1000)

# Entraîner
log_reg.fit(X_train, y_train)

# Predire
y_pred = log_reg.predict(X_test)

# Probabilites
y_proba = log_reg.predict_proba(X_test)
print(y_proba[0]) # Exemple: [0.05, 0.70, 0.25]

# Evaluer
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy*100:.2f}%")
```

Régression Logistique : Avantages

Avantages

- Très rapide
- Donne probabilités
- Bon en binaire
- Peu de paramètres
- Interprétable

Inconvénients

- Linéaire seulement
- Moins bon si non-linéaire
- Besoin normalisation

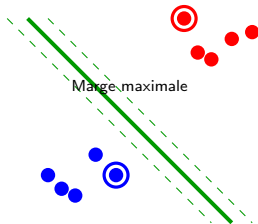
Quand l'utiliser ?

Classification binaire simple, besoin de probabilités, rapidité

SVM : Principe

L'idée

Trouver la **meilleure frontière** qui sépare les classes avec la plus grande marge



Objectif : Maximiser la distance entre les classes

Définition

Les **support vectors** sont les points les plus proches de la frontière

- Ce sont les points les plus importants
- Seuls ces points influencent la frontière
- Supprimer les autres points ne change rien !

Pourquoi SVM ?

Support Vector Machine = Machine à Vecteurs de Support
→ Nommée d'après ces points critiques

Problème

Et si les données ne sont pas linéairement séparables ?

Solution : Kernels

Transformer les données dans un espace de dimension supérieure

Kernels populaires :

- **linear** : Pour données linéaires (rapide)
- **rbf** (radial) : Pour données non-linéaires (par défaut)
- **poly** : Polynomial
- **sigmoid** : Comme réseau de neurones

SVM : Code Python

```
from sklearn.svm import SVC

# Creer le modele SVM
svm = SVC(
    kernel='rbf',      # Kernel (rbf, linear, poly)
    C=1.0,             # Regularisation
    gamma='scale'      # Kernel coefficient
)

# Entrainer
svm.fit(X_train, y_train)

# Predire
y_pred = svm.predict(X_test)

# Evaluer
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy*100:.2f}%")
```


Paramètres importants

- **C** : Contrôle le compromis marge/erreurs
 - C petit $\rightarrow$ grande marge, plus d'erreurs (underfitting)
 - C grand $\rightarrow$ petite marge, moins d'erreurs (overfitting)
- **kernel** : Type de transformation
- **gamma** : Pour rbf/poly kernels
 - gamma petit $\rightarrow$ influence large
 - gamma grand $\rightarrow$ influence locale (overfitting)

SVM : Avantages et Inconvénients

Avantages

- Très efficace
- Bon en haute dimension
- Versatile (kernels)
- Robuste

Inconvénients

- Lent (gros datasets)
- Sensible aux paramètres
- Difficile à interpréter
- Pas de probabilités directes

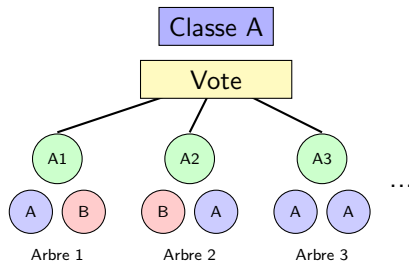
Quand l'utiliser ?

Datasets moyens, haute dimension, frontières complexes

Random Forests : Principe

L'idée

Combiner **plusieurs arbres** pour être plus robuste et précis



Principe : Beaucoup d'arbres votent ensemble

Comment ça marche ?

Algorithme

- ① Créer N arbres différents
- ② Chaque arbre est entraîné sur :
 - Un sous-ensemble aléatoire des données (bootstrap)
 - Un sous-ensemble aléatoire des features
- ③ Pour prédire : chaque arbre vote
- ④ Classe finale = vote majoritaire

Pourquoi ça marche ?

- Réduction variance (overfitting)
- “La sagesse des foules”
- Erreurs se compensent

Random Forests : Code

```
from sklearn.ensemble import RandomForestClassifier

# Creer la foret
rf = RandomForestClassifier(
    n_estimators=100,      # Nombre d'arbres
    max_depth=10,         # Profondeur max
    min_samples_split=2,
    random_state=42
)

# Entrainer
rf.fit(X_train, y_train)

# Predire
y_pred = rf.predict(X_test)

# Evaluer
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy*100:.2f}%")
```

Paramètres importants

- **n_estimators** : Nombre d'arbres
 - Plus → meilleur, mais plus lent
 - Typique : 100-500
- **max_depth** : Profondeur des arbres
- **max_features** : Nombre de features par arbre
 - 'sqrt' → racine carrée (par défaut)
 - 'log2' → logarithme base 2
- **min_samples_split/leaf** : Contrôle complexité

Random Forests : Avantages

Avantages

- Très performant
- Résiste à l'overfitting
- Gère bien données manquantes
- Indique importance features
- Fonctionne bien "out of the box"
- Gère grandes dimensions
- Parallélisable

Inconvénients

- Moins interprétable qu'un arbre
- Plus lent qu'un arbre
- Plus de mémoire

Quand utiliser Random Forests ?

Excellent choix quand :

- Besoin de bonnes performances
- Dataset moyen/grand
- Pas besoin d'interprétabilité extrême
- Besoin de robustesse
- Première approche sur nouveau problème

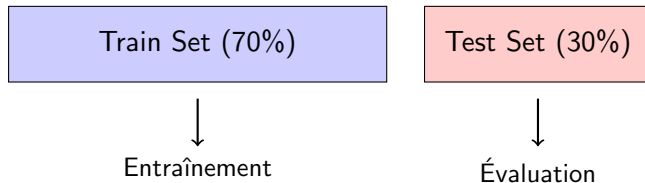
Conseil

Random Forests est souvent le **meilleur point de départ** pour un problème de classification !

Train/Test Split

Règle d'or

JAMAIS évaluer sur les données d'entraînement !



Proportions courantes :

- 70/30, 80/20, 75/25

Accuracy

Définition

Pourcentage de prédictions correctes

$$Accuracy = \frac{\text{Bonnes prédictions}}{\text{Total}}$$

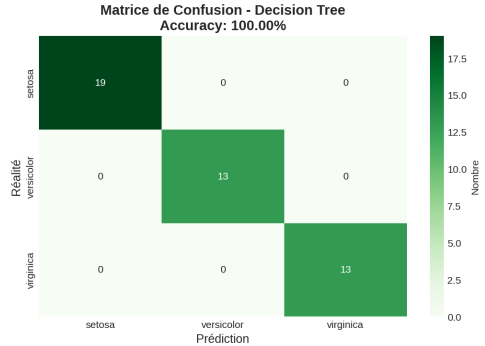
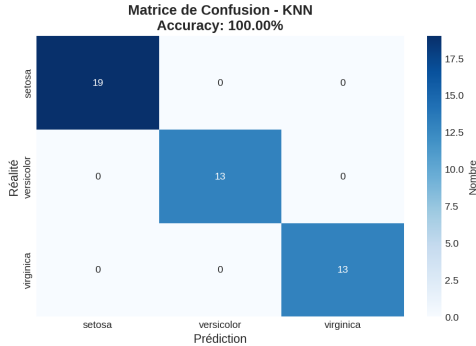
Exemple

100 prédictions, 95 correctes → Accuracy = 95%

Attention !

Peut être trompeur si classes déséquilibrées !

Matrice de Confusion



Visualise toutes les prédictions vs réalité

Précision

$$\frac{VP}{VP+FP}$$

Parmi prédictions positives, combien vraies ?

Rappel

$$\frac{VP}{VP+FN}$$

Parmi vrais positifs, combien détectés ?

F1-Score

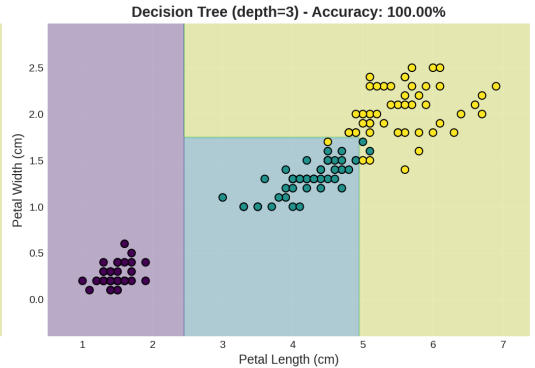
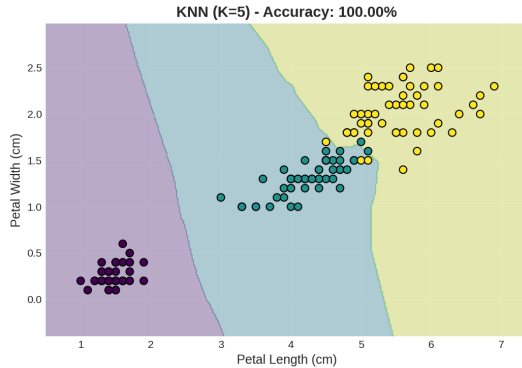
$$2 \times \frac{P \times R}{P + R}$$

Moyenne harmonique

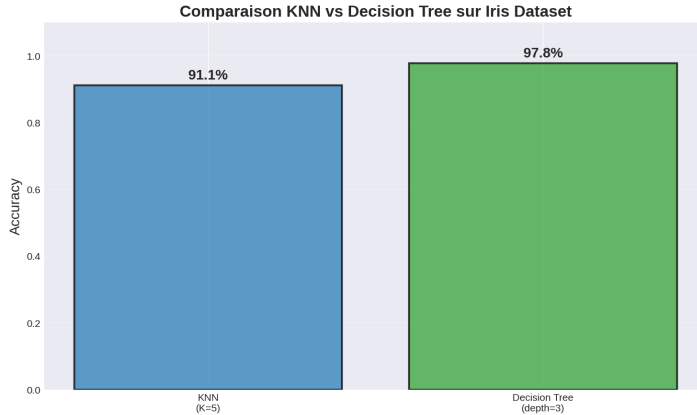
Utilité

Équilibre précision/rappel

Essentiel pour classes déséquilibrées



Performance Comparative



Quel Modèle Choisir ?

Situation	Modèle
Petit dataset, besoin simplicité	KNN
Besoin interprétabilité	Arbres
Classification binaire simple	Log. Reg.
Haute dimension, non-linéaire	SVM
Meilleure performance générale	Random Forest

Conseil

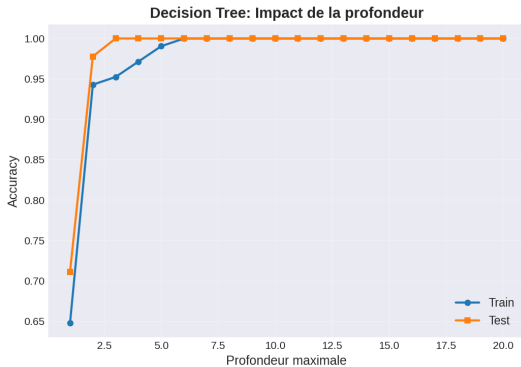
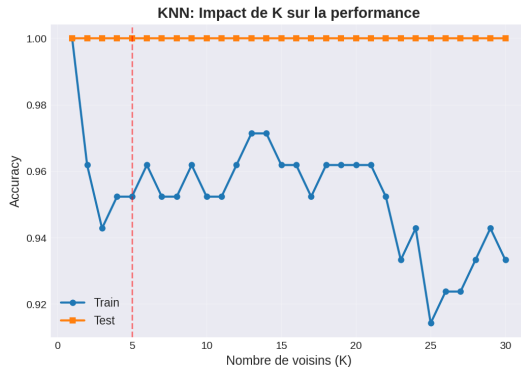
Essayez plusieurs modèles et comparez !

Tableau Récapitulatif

	KNN	Arbres	Log Reg	SVM	RF
Simplicité	+++	+++	++	+	++
Performance	++	++	++	+++	+++
Rapidité	+	+++	+++	+	++
Interprétable	++	+++	++	+	+
Gros datasets	+	++	+++	+	++

+ = faible, ++ = moyen, +++ = excellent

Éviter l'Overfitting



Règle

Surveiller Train vs Test accuracy

Stratégies Anti-Overfitting

Pour KNN

- Augmenter K
- Normaliser données
- Réduire features

Pour Arbres/RF

- Limiter max_depth
- Augmenter min_samples_split
- Pruning (élagage)

Pour tous

- Plus de données
- Cross-validation
- Régularisation

Exemple Complet : Comparer Tous les Modèles

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Charger et preparer
iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.3, random_state=42)

# Creer les modeles
models = {
    'KNN': KNeighborsClassifier(n_neighbors=5),
    'Tree': DecisionTreeClassifier(max_depth=3),
    'LogReg': LogisticRegression(max_iter=1000),
    'SVM': SVC(kernel='rbf'),
    'RF': RandomForestClassifier(n_estimators=100)
}
```

Exemple Complet (suite)

```
# Entraîner et évaluer chaque modele
results = {}
for name, model in models.items():
    # Entraîner
    model.fit(X_train, y_train)

    # Predire
    y_pred = model.predict(X_test)

    # Evaluer
    acc = accuracy_score(y_test, y_pred)
    results[name] = acc

    print(f"{name}: {acc*100:.2f}%")

# Meilleur modele
best = max(results, key=results.get)
print(f"\nMeilleur: {best} ({results[best]*100:.2f}%")
```

Workflow Recommandé

- 1 **Explorer** les données
- 2 **Préparer** : nettoyage, normalisation
- 3 **Diviser** Train/Test
- 4 **Tester** plusieurs modèles simples
- 5 **Comparer** performances
- 6 **Optimiser** le meilleur
- 7 **Valider** sur test set
- 8 **Analyser** erreurs

Important

Ne jamais optimiser sur le test set !

Normalisation : Quand et Comment ?

Modèles qui en ont BESOIN :

- KNN (distances !)
- SVM
- Régression Logistique

Modèles qui n'en ont PAS besoin :

- Arbres de Décision
- Random Forests

Méthode **StandardScaler** :

- Moyenne = 0
- Écart-type = 1

Normalisation : Code

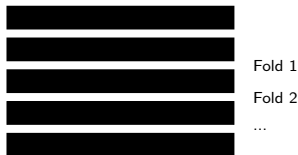
```
1 from sklearn.preprocessing import StandardScaler
2
3 # Creer le scaler
4 scaler = StandardScaler()
5
6 # IMPORTANT: fit sur train uniquement!
7 scaler.fit(X_train)
8
9 # Transform train et test
10 X_train_scaled = scaler.transform(X_train)
11 X_test_scaled = scaler.transform(X_test)
12
13 # Utiliser les donnees normalisees
14 knn = KNeighborsClassifier(n_neighbors=5)
15 knn.fit(X_train_scaled, y_train)
16 y_pred = knn.predict(X_test_scaled)
```

Problème

Un seul train/test split peut être chanceux ou malchanceux

Solution : K-Fold Cross-Validation

- 1 Diviser données en K parts
- 2 Pour chaque part :
 - Utiliser comme test
 - Reste = train
- 3 Moyenne des K résultats



Cross-Validation : Code

```
from sklearn.model_selection import cross_val_score

# Modele
model = RandomForestClassifier(n_estimators=100)

# Cross-validation avec 5 folds
scores = cross_val_score(
    model, X, y,
    cv=5,          # 5 folds
    scoring='accuracy'
)

print(f"Scores: {scores}")
print(f"Moyenne: {scores.mean():.2f}")
print(f"Ecart-type: {scores.std():.2f}")
```

Avantages :

- Estimation plus robuste
- Utilise toutes les données
- Détecte instabilité

Exercice 1 : Wine Dataset

Objectif

Classifier 3 types de vins selon leurs caractéristiques chimiques

À faire :

- 1 Charger : `from sklearn.datasets import load_wine`
- 2 Explorer : shape, features, classes
- 3 Train/Test split (70/30)
- 4 Tester KNN, Arbre, Random Forest
- 5 Comparer accuracies
- 6 Afficher matrices de confusion
- 7 Identifier meilleur modèle

Exercice 2 : Optimisation

Objectif

Trouver les meilleurs hyperparamètres

À faire :

- 1 KNN : tester K de 1 à 20
- 2 Créer graphique accuracy vs K
- 3 Arbres : tester max_depth de 1 à 10
- 4 Random Forest : tester n_estimators
- 5 Identifier valeurs optimales
- 6 Comparer avec paramètres par défaut

Checklist des Bonnes Pratiques

- 1 Toujours diviser Train/Test AVANT tout
- 2 Normaliser quand nécessaire (KNN, SVM, LogReg)
- 3 Ne JAMAIS toucher au test set pendant développement
- 4 Tester plusieurs modèles
- 5 Utiliser plusieurs métriques (pas juste accuracy)
- 6 Vérifier overfitting (train vs test)
- 7 Cross-validation pour robustesse
- 8 Visualiser résultats (confusion matrix)
- 9 Documenter code et choix
- 10 Comprendre les erreurs du modèle

Quand Utiliser Chaque Modèle ?

- **KNN** : Petit dataset, peu features, prototype rapide
- **Arbres** : Besoin interprétabilité, features mix
- **Log. Reg.** : Binaire simple, besoin probabilités
- **SVM** : Haute dimension, frontières complexes
- **Random Forests** : Point de départ, besoin performance

Récapitulatif : 5 Algorithmes

Vous avez appris :

- ➊ **KNN** : Vote des K voisins
- ➋ **Arbres** : Questions successives
- ➌ **Régression Logistique** : Probabilités via sigmoïde
- ➍ **SVM** : Marge maximale
- ➎ **Random Forests** : Ensemble d'arbres

Concepts essentiels :

- Train/Test split
- Métriques : Accuracy, Précision, Rappel, F1
- Overfitting vs Underfitting
- Normalisation
- Cross-validation

Documentation

- Scikit-learn : scikit-learn.org
- Cours Andrew Ng : Machine Learning (Coursera)
- Livre : “Hands-On ML” (Géron)

Datasets pour pratiquer

- Scikit-learn : `load_iris`, `load_wine`, `load_breast_cancer`
- Kaggle : kaggle.com/datasets
- UCI ML Repository